Optimization-Driven Tuning of LLM Inference Configurations

Group Members: 1, 2. , 3.

Problem, motivation, and scope

The main idea of this project is to find the best way to run a large language model on a given machine or hardware. In practice, there are multiple choices when serving an LLM: how much to compress the model, how many requests to process together, how to split work across GPUs, how much memory to reserve, and how to schedule incoming requests. The difficulty is that there is no single setting that works best in every situation. A configuration that is fast may use too much of RAM, a cheap configuration may reduce accuracy, and an energy efficient configuration may increase response time. The application context is simple: In a real deployment, serving decisions cannot be made by changing one parameter knob at a time.

At the same time, we do not want to make the project too broad. We are not trying to build a universal optimizer over every possible inference parameter. Instead, we will study a compact decision vector $x = (q, b, p)$, where q denotes the precision or quantization choice, b captures batching or maximum-token settings, and p captures a small parallelism choice such as tensor-parallel degree or a limited multi-GPU placement option.

Optimization formulations that motivate the study

Three formulations from different studies help frame the project. They give us three different optimization viewpoints: constrained black-box search, linear programming over memory placement, and network-flow based MILP for distributed placement.

1. Vidur-style constrained deployment search (Agrawal et al., 2024): Vidur-Search sees deployment tuning as a constrained search problem: find the configuration that satisfies latency requirements but also is cost effective. A simplified version of that idea is:

$$\max_{x \in \mathcal{X}} \frac{\mathrm{QPS}_{\max}(x; w)}{\mathrm{Cost}(x)} \tag{1}$$

$$\text{subject to} \quad \mathrm{TTFT}_p(x; w) \le \tau_1, \qquad \mathrm{TBT}_q(x; w) \le \tau_2, \qquad \mathrm{Delay}_{0.99}(x; w) \le \tau_3. \tag{2}$$

Here x is a deployment configuration and w is the workload trace. TTFT is Time To First Token, TBT is Time Between Tokens, Delay(0.99) is the total response time for a request, measured at the 99th percentile so it captures latency in worst case scenario. The important idea: maximize queries per dollar while respecting service level objectives.

2. FlexGen-style linear programming for offloading (Sheng et al., 2023): FlexGen shows that once the execution schedule is fixed, tensor placement across GPU, CPU, and disk can be written as a small linear program:

$$\min_{p} \frac{T}{b\,l\,s} \tag{3}$$

$$\text{subject to} \quad \text{GPU}_{\text{peak_mem}} \leq \text{GPU}_{\text{cap}}, \qquad \text{CPU}_{\text{peak_mem}} \leq \text{CPU}_{\text{cap}}, \qquad \text{Disk}_{\text{peak_mem}} \leq \text{Disk}_{\text{cap}}, \tag{4}$$

$$w_g + w_c + w_d = 1, \qquad c_g + c_c + c_d = 1, \qquad h_g + h_c + h_d = 1. \tag{5}$$

In this formulation, p represents the fractions of weights, KV-cache, and activations stored at each memory tier. This formulation is useful because it separates out one part of inference tuning that is structured enough to model analytically.

3. Helix-style max-flow and MILP for multi-GPU placement (Mei et al., 2025): Helix models a heterogeneous cluster as a directed graph and searches for a placement that maximizes feasible flow through the system:

$$\max_{f,\Psi} \sum_i f_{\text{source},i} \tag{6}$$

$$\text{subject to} \quad \sum_u f_{u,i} = \sum_v f_{i,v}, \tag{7}$$

$$\sum_u f_{u,i} \leq \sum_{j=1}^{k} b_i^j T_j, \qquad f_{i,j} \leq d_{i,j} S_{i,j}. \tag{8}$$

The variables f describe flow through compute nodes and network links, while the placement variables encode which layers live on which GPUs. This is a much more combinatorial problem than the earlier one, and it is useful to us because it shows how placement and scheduling can be optimized jointly rather than sequentially.

Methodology, goals, and expected outcomes

Our approach is based around comparison, not on claiming one grand objective for every possible deployment scenario. We will start by fixing a serving stack and one or two example workloads, then measure a bounded set of configurations over the chosen variables q, b, and p. Those measurements will give us latency, throughput, memory use, and possibly energy estimates.

The expected outcome is to create a small but clear optimization benchmark for LLM inference tuning, carefully compare optimization algorithms on that benchmark, and then a final recommendation for one deployment scenario based on trade-off curves.

Datasets

- **Dataset 1** — ShareGPT_Vicuna_unfiltered *(HuggingFace)*
- **Dataset 2** — Vidur Synthetic Workload Traces *(Microsoft GitHub)*

References

Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. *VIDUR: A Large-Scale Simulation Framework for LLM Inference*. Proceedings of Machine Learning and Systems, 2024.

Mei, Y., Zhuang, Y., Miao, X., Yang, J., Jia, Z., and Vinayak, R. *Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow*. ASPLOS, 2025.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Re, C., Stoica, I., and Zhang, C. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU*. ICML, 2023.